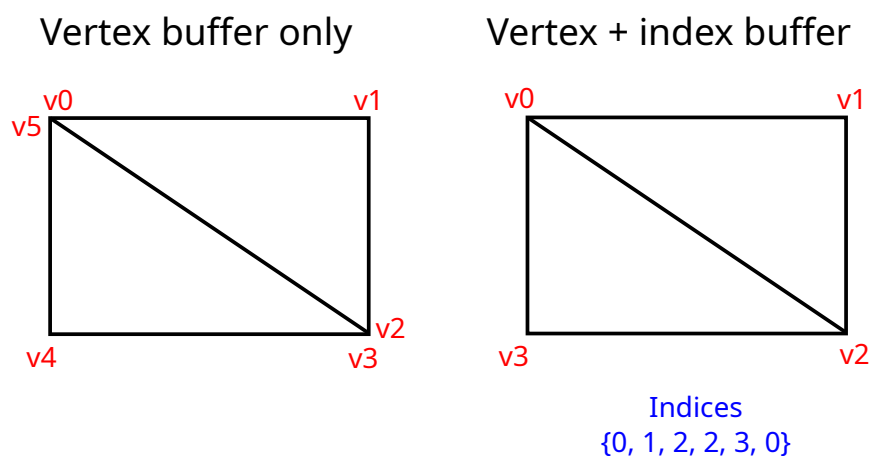


Index buffer

- [Introduction](#)
- [Index buffer creation](#)
- [Using an index buffer](#)

Introduction

The 3D meshes you'll be rendering in a real world application will often share vertices between multiple triangles. This already happens even with something simple like drawing a rectangle:



Drawing a rectangle takes two triangles, which means that we need a vertex buffer with 6 vertices. The problem is that the data of two vertices needs to be duplicated resulting in 50% redundancy. It only gets worse with more complex meshes, where vertices are reused in an average number of 3 triangles. The solution to this problem is to use an *index buffer*.

An index buffer is essentially an array of pointers into the vertex buffer. It allows you to reorder the vertex data, and reuse existing data for multiple vertices. The illustration above demonstrates what the index buffer would look like for the rectangle if we have a vertex buffer containing each of the four unique vertices. The first three indices define the upper-right triangle and the last three indices define the vertices for the bottom-left triangle.

Index buffer creation

In this chapter we're going to modify the vertex data and add index data to draw a rectangle like the one in the illustration. Modify the vertex data to represent the four corners:

```
const std::vector<Vertex> vertices = {
    {{-0.5f, -0.5f}, {1.0f, 0.0f, 0.0f}},
    {{0.5f, -0.5f}, {0.0f, 1.0f, 0.0f}},
    {{0.5f, 0.5f}, {0.0f, 0.0f, 1.0f}},
    {{-0.5f, 0.5f}, {1.0f, 1.0f, 1.0f}}
};
```

The top-left corner is red, top-right is green, bottom-right is blue and the bottom-left is white. We'll add a new array `indices` to represent the contents of the index buffer. It should match the indices in the illustration to draw the upper-right triangle and bottom-left triangle.

```
const std::vector<uint16_t> indices = {
    0, 1, 2, 2, 3, 0
};
```

It is possible to use either `uint16_t` or `uint32_t` for your index buffer depending on the number of entries in `vertices`. We can stick to `uint16_t` for now because we're using less than 65535 unique vertices.

Just like the vertex data, the indices need to be uploaded into a `VkBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkBuffer.html>) for the GPU to be able to access them. Define two new class members to hold the resources for the index buffer:

```
VkBuffer vertexBuffer;
VkDeviceMemory vertexBufferMemory;
VkBuffer indexBuffer;
VkDeviceMemory indexBufferMemory;
```

The `createIndexBuffer` function that we'll add now is almost identical to `createVertexBuffer`:

```

void initVulkan() {
    ...
    createVertexBuffer();
    createIndexBuffer();
    ...
}

void createIndexBuffer() {
    VkDeviceSize bufferSize = sizeof(indices[0]) * indices.size();

    VkBuffer stagingBuffer;
    VkDeviceMemory stagingBufferMemory;
    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_SRC_BIT, VK_MEMORY_PROPERTY_HOST_
VISIBLE_BIT | VK_MEMORY_PROPERTY_HOST_COHERENT_BIT, stagingBuffer, stagingBufferMemor
y);

    void* data;
    vkMapMemory(device, stagingBufferMemory, 0, bufferSize, 0, &data);
    memcpy(data, indices.data(), (size_t) bufferSize);
    vkUnmapMemory(device, stagingBufferMemory);

    createBuffer(bufferSize, VK_BUFFER_USAGE_TRANSFER_DST_BIT | VK_BUFFER_USAGE_INDEX_B
UFFER_BIT, VK_MEMORY_PROPERTY_DEVICE_LOCAL_BIT, indexBuffer, indexBufferMemory);

    copyBuffer(stagingBuffer, indexBuffer, bufferSize);

    vkDestroyBuffer(device, stagingBuffer, nullptr);
    vkFreeMemory(device, stagingBufferMemory, nullptr);
}

```

There are only two notable differences. The `bufferSize` is now equal to the number of indices times the size of the index type, either `uint16_t` or `uint32_t`. The usage of the `indexBuffer` should be `VK_BUFFER_USAGE_INDEX_BUFFER_BIT` instead of `VK_BUFFER_USAGE_VERTEX_BUFFER_BIT`, which makes sense. Other than that, the process is exactly the same. We create a staging buffer to copy the contents of `indices` to and then copy it to the final device local index buffer.

The index buffer should be cleaned up at the end of the program, just like the vertex buffer:

```

void cleanup() {
    cleanupSwapChain();

    vkDestroyBuffer(device, indexBuffer, nullptr);
    vkFreeMemory(device, indexBufferMemory, nullptr);

    vkDestroyBuffer(device, vertexBuffer, nullptr);
    vkFreeMemory(device, vertexBufferMemory, nullptr);

    ...
}

```

Using an index buffer

Using an index buffer for drawing involves two changes to `recordCommandBuffer`. We first need to bind the index buffer, just like we did for the vertex buffer. The difference is that you can only have a single index buffer. It's unfortunately not possible to use different indices for each vertex attribute, so we do still have to completely duplicate vertex data even if just one attribute varies.

```
vkCmdBindVertexBuffers(commandBuffer, 0, 1, vertexBuffers, offsets);  
  
vkCmdBindIndexBuffer(commandBuffer, indexBuffer, 0, VK_INDEX_TYPE_UINT16);
```

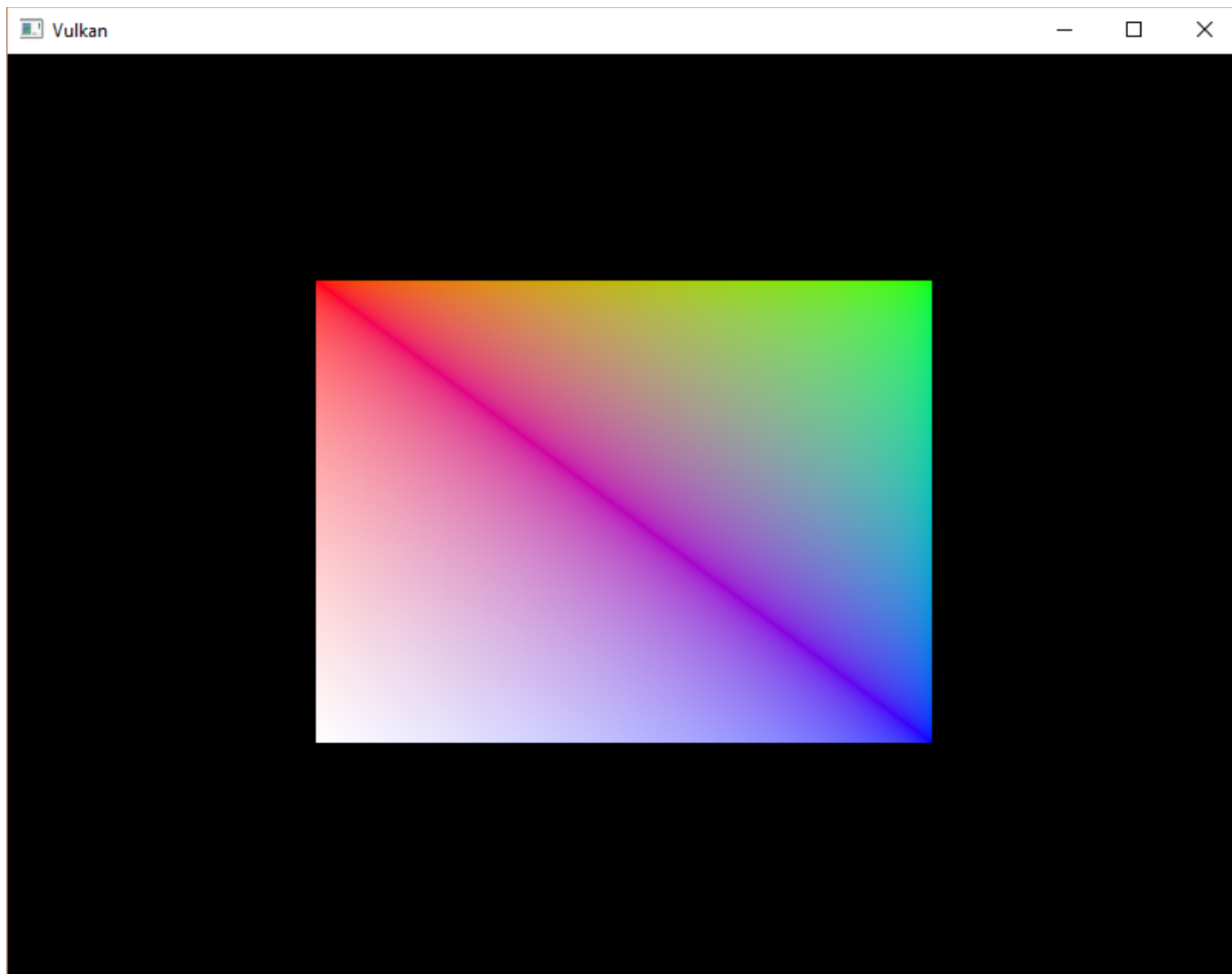
An index buffer is bound with `vkCmdBindIndexBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdBindIndexBuffer.html>) which has the index buffer, a byte offset into it, and the type of index data as parameters. As mentioned before, the possible types are `VK_INDEX_TYPE_UINT16` and `VK_INDEX_TYPE_UINT32`.

Just binding an index buffer doesn't change anything yet, we also need to change the drawing command to tell Vulkan to use the index buffer. Remove the `vkCmdDraw` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdDraw.html>) line and replace it with `vkCmdDrawIndexed` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdDrawIndexed.html>):

```
vkCmdDrawIndexed(commandBuffer, static_cast<uint32_t>(indices.size()), 1, 0, 0, 0);
```

A call to this function is very similar to `vkCmdDraw` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdDraw.html>). The first two parameters specify the number of indices and the number of instances. We're not using instancing, so just specify `1` instance. The number of indices represents the number of vertices that will be passed to the vertex shader. The next parameter specifies an offset into the index buffer, using a value of `1` would cause the graphics card to start reading at the second index. The second to last parameter specifies an offset to add to the indices in the index buffer. The final parameter specifies an offset for instancing, which we're not using.

Now run your program and you should see the following:



You now know how to save memory by reusing vertices with index buffers. This will become especially important in a future chapter where we're going to load complex 3D models.

The previous chapter already mentioned that you should allocate multiple resources like buffers from a single memory allocation, but in fact you should go a step further. [Driver developers recommend \(https://developer.nvidia.com/vulkan-memory-management\)](https://developer.nvidia.com/vulkan-memory-management) that you also store multiple buffers, like the vertex and index buffer, into a single `VkBuffer` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/VkBuffer.html>) and use offsets in commands like `vkCmdBindVertexBuffers` (<https://www.khronos.org/registry/vulkan/specs/1.0/man/html/vkCmdBindVertexBuffers.html>). The advantage is that your data is more cache friendly in that case, because it's closer together. It is even possible to reuse the same chunk of memory for multiple resources if they are not used during the same render operations, provided that their data is refreshed, of course. This is known as *aliasing* and some Vulkan functions have explicit flags to specify that you want to do this.

C++ code ([../code/21_index_buffer.cpp](#)) / Vertex shader ([../code/18_shader_vertexbuffer.vert](#)) / Fragment shader ([../code/18_shader_vertexbuffer.frag](#))

[Previous \(/en/Vertex_buffers/Staging_buffer\)](#)

[Next \(/en/Uniform_buffers/Descriptor_set_layout_and_buffer\)](#)